

BENCHMARKING DE DESEMPENHO:

PROCESSOS VS. THREADS EM JAVA

Disciplina: Sistemas Operacionais | Análise Técnica

Ambiente: Linux | Java 21 | CPU com 12 núcleos lógicos

Matriz: 1.000 × 1.000 doubles | 5 execuções por cenário (1 descartada como aquecimento)

1. QUAL ABORDAGEM FOI MAIS RÁPIDA? POR QUÊ?

As threads foram significativamente mais rápidas em todos os cenários testados. Com 8 workers, a versão baseada em threads concluiu a multiplicação em **54,78 ms** em média, enquanto a versão com processos levou **519,74 ms** — uma diferença de aproximadamente **9,5 vezes**. A tabela abaixo resume os resultados coletados:

Tipo	Workers	Média (ms)	Speedup*
Threads	2	150,15	1,00×
Threads	4	79,21	1,90×
Threads	8	54,78	2,74×
Processos	2	402,22	0,37×
Processos	4	423,61	0,35×
Processos	8	519,74	0,29×

* Speedup relativo ao baseline Threads com 2 workers (150,15 ms).

A causa principal dessa diferença está no custo de criação de cada processo. No modelo implementado com `ProcessBuilder`, o processo pai precisa lançar N instâncias

completamente novas da JVM. Cada uma delas passa pelo ciclo completo de inicialização: carregamento do bytecode, alocação do heap, inicialização do Garbage Collector e compilação JIT a frio. Esse overhead de startup fica entre **100 e 200 ms por processo**, independentemente do trabalho computacional a ser feito.

Além do startup, o modelo de processos exige comunicação explícita entre pai e filhos (IPC). Como processos não compartilham memória, as matrizes A e B (16 MB no total para $N=1.000$) precisam ser serializadas em um arquivo binário temporário a cada execução — e cada processo filho lê esse arquivo do disco individualmente antes de começar a calcular. Esse custo de I/O está embutido em todo tempo medido.

Já no modelo de threads, A e B existem uma única vez no heap da JVM. Cada thread recebe apenas uma referência para esses objetos e começa a calcular imediatamente, sem serialização e sem nenhum custo de inicialização extra. O ganho de desempenho das threads é, portanto, estrutural: elas eliminam dois dos maiores custos do modelo de processos (startup de JVM e IPC).

2. QUAL CONSUMIU MAIS MEMÓRIA? POR QUÊ?

A versão com processos consome muito mais memória do sistema, e o crescimento é multiplicativo com o número de workers. Para entender o porquê, é preciso comparar como cada modelo gerencia o heap.

No modelo de threads, todas elas vivem dentro da **mesma instância da JVM**. As matrizes A, B e C são alocadas uma única vez no heap compartilhado. Com $N=1.000$ e double de 8 bytes, cada matriz ocupa 8 MB — ou seja, o conjunto completo de dados do problema ocupa ~24 MB, independentemente de quantas threads estejam rodando. A única memória adicional por thread é sua própria pilha de execução (stack), que para esse problema é da ordem de alguns kilobytes.

No modelo de processos, cada filho é uma JVM completamente independente com seu próprio heap isolado. Cada processo filho carrega as duas matrizes de entrada A e B do arquivo para a sua própria memória — mesmo que outro processo já as tenha carregado. Com 8 processos, o sistema mantém simultaneamente 8 cópias de A e 8 cópias de B na RAM, além do overhead base de cada JVM (tipicamente 30 a 60 MB de memória nativa para estruturas internas do runtime). O consumo total pode facilmente ultrapassar **500 MB** para um cenário com 8 workers.

Esse isolamento de heap é um reflexo direto do modelo de processos do sistema operacional: cada processo possui seu próprio espaço de endereçamento virtual, protegido pelo kernel. Não há como duas JVMs acessarem a mesma região de memória por padrão sem recorrer a mecanismos específicos como memória compartilhada (POSIX *shared memory*) ou arquivos mapeados em memória — recursos que não foram utilizados nesta implementação exatamente para representar o comportamento natural do modelo.

3. QUAL APRESENTOU MAIOR ISOLAMENTO?

O modelo de processos oferece isolamento muito superior. Por definição, cada JVM filha é um processo independente do ponto de vista do sistema operacional: possui seu próprio espaço de endereçamento, descritores de arquivo e estado de execução. O kernel garante que nenhum processo consiga ler ou escrever diretamente na memória de outro — qualquer acesso cruzado exigiria chamadas de sistema explícitas, que o SO pode inspecionar e bloquear.

Na prática, isso significa que se um processo filho travar, lançar uma exceção não tratada ou ser encerrado abruptamente pelo SO (por exemplo, por violação de segmento ou por atingir um limite de memória), os demais processos continuam operando normalmente. O processo pai detecta a falha via `waitFor()` com exit code diferente de zero e pode reagir — relançar o worker, registrar o erro ou abortar a operação de forma controlada.

No modelo de threads, o isolamento é muito menor. Todas as threads compartilham o mesmo heap e o mesmo processo do SO. Uma exceção do tipo `OutOfMemoryError`, ou um bug que corrompa a matriz C, afeta todas as threads simultaneamente. Se a JVM travar — por exemplo, por um deadlock que o runtime decida encerrar — a aplicação inteira cai. Essa fragilidade é o trade-off direto do compartilhamento de memória que torna as threads mais rápidas.

4. O GANHO DE DESEMPENHO FOI LINEAR AO AUMENTAR AS UNIDADES DE EXECUÇÃO?

Para as threads, o escalonamento foi **sublinear, mas próximo do ideal nos primeiros níveis**. De 2 para 4 threads, o speedup foi de **1,90×** (próximo do teórico 2,00×), e de 4 para 8 threads, o ganho adicional foi de apenas **1,45×** em vez dos 2,00× esperados em um cenário ideal. Isso pode ser explicado por três fatores:

- **Saturação de largura de banda de memória:** a multiplicação de matrizes é uma operação memory-bound — o gargalo não é a capacidade de cálculo dos núcleos, mas a velocidade com que a RAM consegue fornecer dados. A partir de certo ponto, adicionar mais threads não acelera o processamento porque todas competem pelo mesmo barramento de memória.
- **Lei de Amdahl:** mesmo que a parte paralela (cálculo das linhas) seja enorme, existe uma fração serial do programa — geração das matrizes, montagem do resultado, sincronização via `CountDownLatch` — que limita matematicamente o speedup máximo alcançável.
- **Overhead de escalonamento:** com 8 threads em uma CPU de 12 núcleos lógicos, o SO ainda precisa gerenciar os context switches entre threads, especialmente quando o scheduler migra uma thread entre núcleos físicos distintos.

Para os processos, o comportamento foi o oposto: o tempo aumentou com mais workers.

Com 2 processos a média foi 402,22 ms; com 8 processos subiu para 519,74 ms. Isso ocorre porque o overhead fixo por processo (startup de JVM + leitura dos 16 MB do arquivo) domina completamente o tempo de computação. Ao dobrar o número de processos, não se divide o trabalho de forma eficiente — soma-se o custo de inicialização de mais JVMs e a contenção de I/O no disco para leitura paralela do mesmo arquivo de entrada.

5. EM QUAIS CENÁRIOS ARQUITETURAIS USAR PROCESSOS PODE SER MAIS VANTAJOSO?

Apesar do desempenho inferior neste benchmark específico, o modelo de processos é a escolha correta em diversas situações arquiteturais relevantes:

- **Microsserviços e sistemas distribuídos:** quando os workers precisam rodar em máquinas diferentes, threads simplesmente não são uma opção — processos (ou containers) são a unidade natural de distribuição. O custo de IPC que penalizou o benchmark é absorvido pela rede de qualquer forma nesses cenários.
- **Isolamento de segurança e multi-tenancy:** em plataformas SaaS onde código de múltiplos clientes roda no mesmo servidor, o isolamento de espaço de endereçamento é mandatório. Um bug ou ataque em um processo cliente não pode vazar dados de outro cliente — algo que threads no mesmo heap jamais garantiriam.
- **Tolerância a falhas e reinicialização independente:** sistemas de processamento em lote (batch) que precisam retomar tarefas individualmente sem derrubar o orquestrador se beneficiam da independência dos processos. Um worker que trava pode ser relançado isoladamente.
- **Workloads de longa duração:** o custo de inicializar uma JVM (~100–200 ms) é irrelevante para um processo que ficará ativo por horas ou dias. Nesse caso, o

benefício do isolamento e da independência de ciclo de vida supera o overhead inicial de startup.

- **Integração com diferentes runtimes ou linguagens:** se partes do sistema são escritas em Python, Rust ou Go, processos são o único mecanismo de integração praticável. Threads só funcionam dentro de uma única linguagem e runtime.

Em resumo, processos são a escolha certa quando *isolamento, resiliência e distribuição* são requisitos prioritários. Threads dominam quando o objetivo é maximizar o throughput em um único nó com dados que cabem na memória compartilhada — exatamente o perfil da multiplicação de matrizes que implementamos neste trabalho.

Código-fonte disponível em: github.com/FelipeWaldrigues/aps-processos-vs-threads-java